GIF Picker Technical Design Document

Application architecture and implementation

November 5, 2026

Purpose and scope

GIF Picker lets users find an animation, preview it, and copy its URL into a chat. GIPHY handles search and media delivery. The app handles the search controls, result pages, and copy feedback.

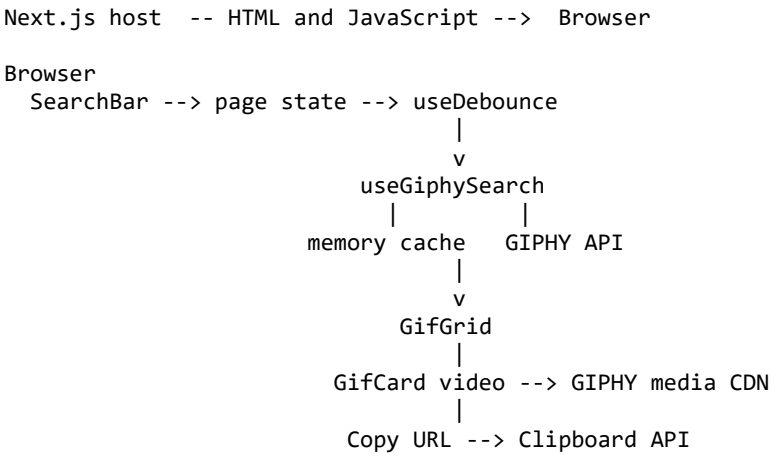
The app uses React, the Next.js App Router, and strict TypeScript. There is no separate backend or database. Next.js serves the page and JavaScript, while the browser calls GIPHY directly. Search history is not saved permanently.

Design constraints

The assignment requires MP4 playback, search through the URL, a usable interface, and a load time below 1.5 seconds. It allows Next.js but limits extra libraries. Native fetch, URLSearchParams, and timers cover the networking and search behavior without adding dependencies.

The assignment lists a limit of 42 requests per hour and 1,000 per day. The app reduces requests by waiting for a pause in typing and reusing pages it has already loaded. It responds to a 429 error rather than counting requests itself. The actual limit comes from the GIPHY account and key.

System boundary



The cache lives in the browser. It is not shared with other users or stored on a server. Video elements load their media directly from the GIPHY URLs in the response.

Implementation status

The application is deployed on Vercel at https://gif-picker-stuller.vercel.app. Production build, TypeScript, automated tests, and live GIPHY integration checks are complete.

Interface examples

These screenshots show the running app with real GIPHY results. The search and copy actions use the same components described in the design.

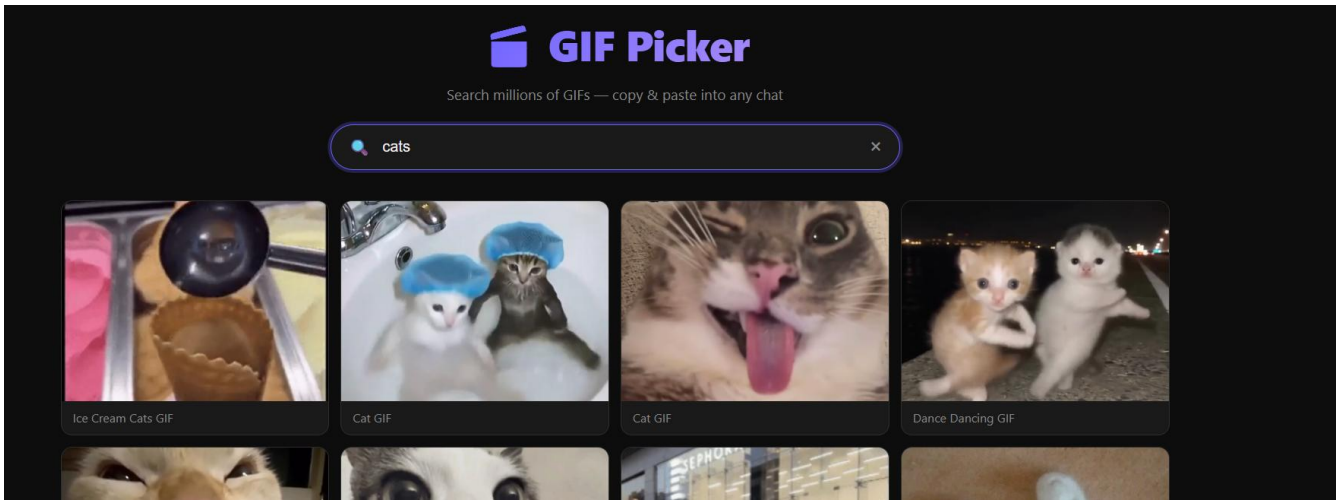


Figure 1 Search results for cats. The address uses q=cats, and Search is available beside the input.

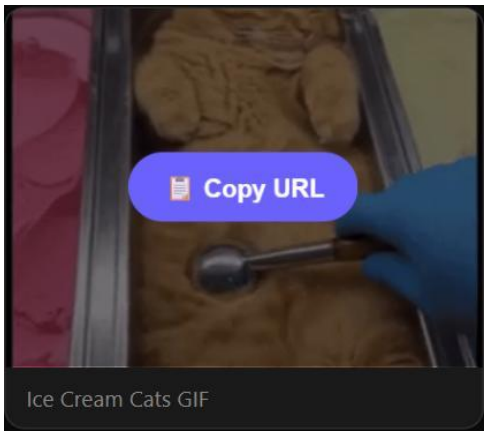


Figure 2 Hovering over a result reveals Copy URL. Touch devices keep the button visible.

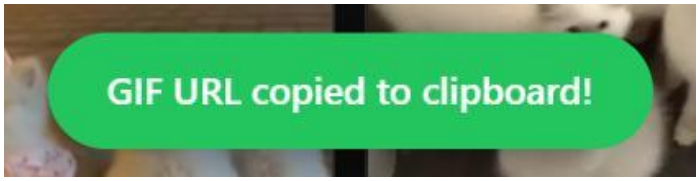


Figure 3 The confirmation appears after the browser clipboard write succeeds.

Components and application state

Component responsibilities

page.tsx brings the screen together. It owns the search, page selection, URL updates, and clipboard message. Components such as GifCard only render their part of the interface and report user actions back to the page.

Module	Responsibility
SearchBar	Controlled input, clear action, and a form that supports Search and Enter.
GifGrid and GifCard	Loading/error/empty views, result cards, MP4 rendering, and copy actions.
Pagination	Previous/Next controls and the current page indicator.
CopyAlert and ApiLimitBanner	Clipboard feedback and provider-quota feedback.
useDebounce	Delayed query updates with an immediate manual-submit path.
useGiphySearch	Request state, cache lookup, retry pause, and stale-response protection.
giphy.ts and cache.ts	Typed API wrappers and a bounded response cache.

State ownership

The text in the search box and the query being fetched are separate values. inputValue changes as the user types. debouncedQuery catches up after a 400 ms pause. Clicking Search or pressing Enter skips that wait, using the same request path as automatic search.

The page index belongs to a particular query. When the query changes, the effective page resets to zero, which is page 1 in the interface. This avoids briefly fetching the old search again while moving to a new one.

Submitting the same query again changes retryCount. This gives the user a way to retry after a network error without editing the text. Successful results still come from the cache, and a rate-limit pause still blocks a new request.

URL synchronization

A link such as /?q=cats opens with cats already in the search box. When a search becomes active, URLSearchParams encodes it and router.replace updates the address. Using replace keeps typing from filling the browser history with intermediate searches. The URL stores the query, not the page number.

Two refs keep the input and address bar from fighting each other. lastWrittenQuery records the query the app last wrote. pendingUrlQuery holds a query received through navigation until the input catches up. If the user starts typing during that change, their new text takes priority.

Users can share or reopen a search link. There is no visible search-history list, and typing a new term does not create a separate back-button entry for that search.

GIPHY integration and media

API boundary

src/lib/giphy.ts keeps the GIPHY request details out of the UI. It builds each request, checks the HTTP status, and returns the response to the hook. The key comes from NEXT_PUBLIC_GIPHY_API_KEY.

Operation	Request parameters	Use
GET /v1/gifs/search	api_key, q, limit=12, offset, rating=g, lang=en	A page of matching GIFs.
GET /v1/gifs/trending	api_key, limit=25, rating=g	The pool used for the initial three animations.

Data contract

```
Response: { data: GiphyGif[], pagination, meta }
pagination: { total_count, count, offset }
GIF: { id, title, url, images }
images.original: full media rendition
images.original_mp4?: MP4 rendition
images.fixed_height?: fixed height rendition
```

The TypeScript interfaces describe the fields the app expects, including optional MP4 renditions. They do not validate JSON at runtime. If GIPHY returns an unexpected structure, the current client may fail when it reads that data.

Search and pagination

Each page contains up to 12 results. The API offset is the zero-based page index multiplied by 12. The total page count comes from pagination.total_count. Prev is disabled on the first page, and Next is disabled on the last calculated page. None of these page numbers represents the request quota.

The app trusts the total returned by GIPHY. It does not separately cap the page count at the provider's maximum offset, so unusually large result sets can reach an offset the API will reject.

Starting results

Before a search, the app fetches up to 25 trending GIFs, shuffles a copy of the array, and takes three. This uses one request instead of three calls to the random endpoint. If fewer than three items come back, only those items can be shown. The selection stays the same while its cache entry is available.

Playback and sharing

GifCard tries original_mp4 first, then fixed_height, then original, using the first non-empty MP4 URL. The video loops, stays muted, and plays inline. There is no image fallback; a result without an MP4 URL has an empty media area.

Copy URL uses images.original.url, falling back to the GIPHY page URL if it is empty. The preview plays MP4, but the copied link is the GIF media URL so it can be shared in chat.

Caching requests and failure handling

Cache design

The cache uses a Map and keeps at most 50 result pages. Each key combines the trimmed, lowercase query with its offset. The starting view uses landing:random, which keeps it separate from anything typed into the search box.

Reading a page moves it to the most recently used end of the Map. When the cache is full, adding a new page removes the least recently used one. Refreshing the browser clears everything. Entries have no expiry timer, although an older page can be removed when the cache reaches capacity.

The 50-page capacity limits memory use; it does not increase the API allowance. The hook looks in the cache before checking the rate-limit pause. That is why a previously loaded page can still open after new requests have been blocked.

Request state

State or event	Behavior
idle	Initial hook state before its effect runs.
loading	Clear displayed GIFs and show skeleton cards while an uncached request runs.
success	Render the response or a no-results message when its data array is empty.
HTTP 429	Show the limit banner and set a one-hour deadline for new requests.
Uncached page during pause	Show the no-cache message. Preserve totals for the same query so a cached page remains reachable.
Other request failure	Show a generic error, clear results/totals, and allow manual retry.

Concurrency and retry policy

Each request effect has an active flag. When the user changes the search or leaves the component, cleanup turns that flag off. A response checks it before updating the screen or cache, so a slow response from an old search cannot replace the current results.

The active flag ignores old responses but does not cancel requests already sent. Those requests may still count against the quota. The API helper accepts an AbortSignal, but the hook does not currently pass one.

After a 429 response, the hook pauses new requests for one hour. It checks the deadline when the search effect runs again; it does not automatically retry when the hour ends. The user can submit again or change pages after the pause. A daily limit may still cause GIPHY to reject the request.

Clipboard handling

The page waits for navigator.clipboard.writeText to succeed before showing a confirmation. If the browser denies access, it shows a failure message instead. Each completed attempt restarts the three-second message timer. Cleanup clears the timer when the page unmounts. Clipboard access needs HTTPS or localhost and browser permission.

Performance and verification

Rendering strategy

A Suspense boundary lets Next.js send the header and disabled search controls before the interactive view is ready. This gives the user something to see early. It does not mean that search or video playback is ready at the same moment.

The interface uses system fonts and plain CSS. Preconnect hints open connections to GIPHY early, and the cards play MP4 instead of animated GIF files. Videos start loading as their cards render. There is no lazy loading, and choosing the original MP4 first may download more data than a smaller preview would.

Measured evidence

Deployed site: https://gif-picker-stuller.vercel.app

Metric	Desktop	Throttled mobile
First Contentful Paint	0.3 s	0.9 s
Largest Contentful Paint	0.7 s	0.9 s
Time to Interactive	0.7 s	1.8 s
Lighthouse performance score	100	93

The supplied performance report records one Edge Lighthouse run per preset with live GIPHY data. First and largest paint meet 1.5 s on both presets; throttled-mobile interactivity measured 1.8 s. Starting videos played in 721 ms in an unthrottled deployed-site run. First Load JavaScript is about 106 kB.

Verification coverage

The current version passes 61 tests across eight Jest suites, a production build, and TypeScript checks. The tests cover requests, debounce, URLs, pagination, caching, late responses, retries, MP4 fallbacks, and clipboard messages.

Completed Edge checks covered search, submission, navigation, retry, pagination, cached results after HTTP 429, clipboard writes, and the 375-pixel layout. The supplied live-site report also records real GIPHY playback, search links, copying, and cached navigation on the Vercel deployment.

Security boundary

NEXT_PUBLIC_GIPHY_API_KEY is visible in browser code, as expected for this client-side integration. It must not contain a private server credential. The source ZIP excludes .env.local. The app has no backend that stores credentials or user data.

The recorded audit found no production-dependency vulnerabilities and six low-severity findings in the Jest development dependency chain. A scoped npm override installs PostCSS 8.5.32 under Next.js, and the production build passed with that version.

Operations and design trade offs

Local execution

Run the project from the folder containing package.json, using Node.js 20 or later. npm ci installs the versions in the lockfile. Create .env.local from .env.example if it does not already exist, then enter the GIPHY key. These commands use Windows PowerShell.

```
npm.cmd ci
# Set NEXT_PUBLIC_GIPHY_API_KEY in .env.local
npm.cmd run dev
```

For a production check, stop the development server and run:

```
npm.cmd test -- --runInBand
npm.cmd run build
npm.cmd start
```

Use development mode while editing. Use the production build for final checks. Rebuild after changing source code or the public API key. outputFileTracingRoot is set to the project directory so an unrelated lockfile in a parent folder does not change the build root.

Hosting

The project is hosted on Vercel as a Next.js app at https://gif-picker-stuller.vercel.app. NEXT_PUBLIC_GIPHY_API_KEY is configured for the production deployment. No database or additional backend setup is required.

Design decisions and operating boundaries

Page-level hooks are enough for this single-screen app and avoid another state-management library. The trade-off is that the page must coordinate the input, URL, retries, and pagination itself. A refresh also loses the cached results.

The implementation uses a trending pool and an in-memory cache to satisfy the assignment behavior and reduce API requests. These choices are documented as project design decisions; they do not establish full alignment with GIPHY provider guidance on caching or result ordering.

Delivery includes the Vercel deployment, live GIPHY integration, desktop and mobile performance measurements, and browser workflow verification. The measured results and operating boundaries are documented in the preceding sections.

References

GIPHY API Documentation: https://developers.giphy.com/docs/api/
GIPHY Integration Guidance: https://developers.giphy.com/docs/api/best-practices/